Machine-Level Programming: Array allocation and access

Arquitectura de Computadores
Departamento de Engenharia Informática
Instituto Superior de Engenharia do Porto

Luís Nogueira (lmn@isep.ipp.pt)

Today

- **Allocation and access**
  - One-dimensional
  - Multi-dimensional (nested)
  - Multi-level (pointers)
- **Access optimizations**
  - Fixed-size arrays

Basic data types

Integral

- Stored and operated on in general (integer) registers
- Signed vs. unsigned depends on instructions used

Intel	Asm	Bytes	C
byte	`b`	1	`[unsigned] char`
word	`w`	2	`[unsigned] short`
double word	`l`	4	`[unsigned] int`
quad word	`q`	8	`[unsigned] long int` (x86-64)

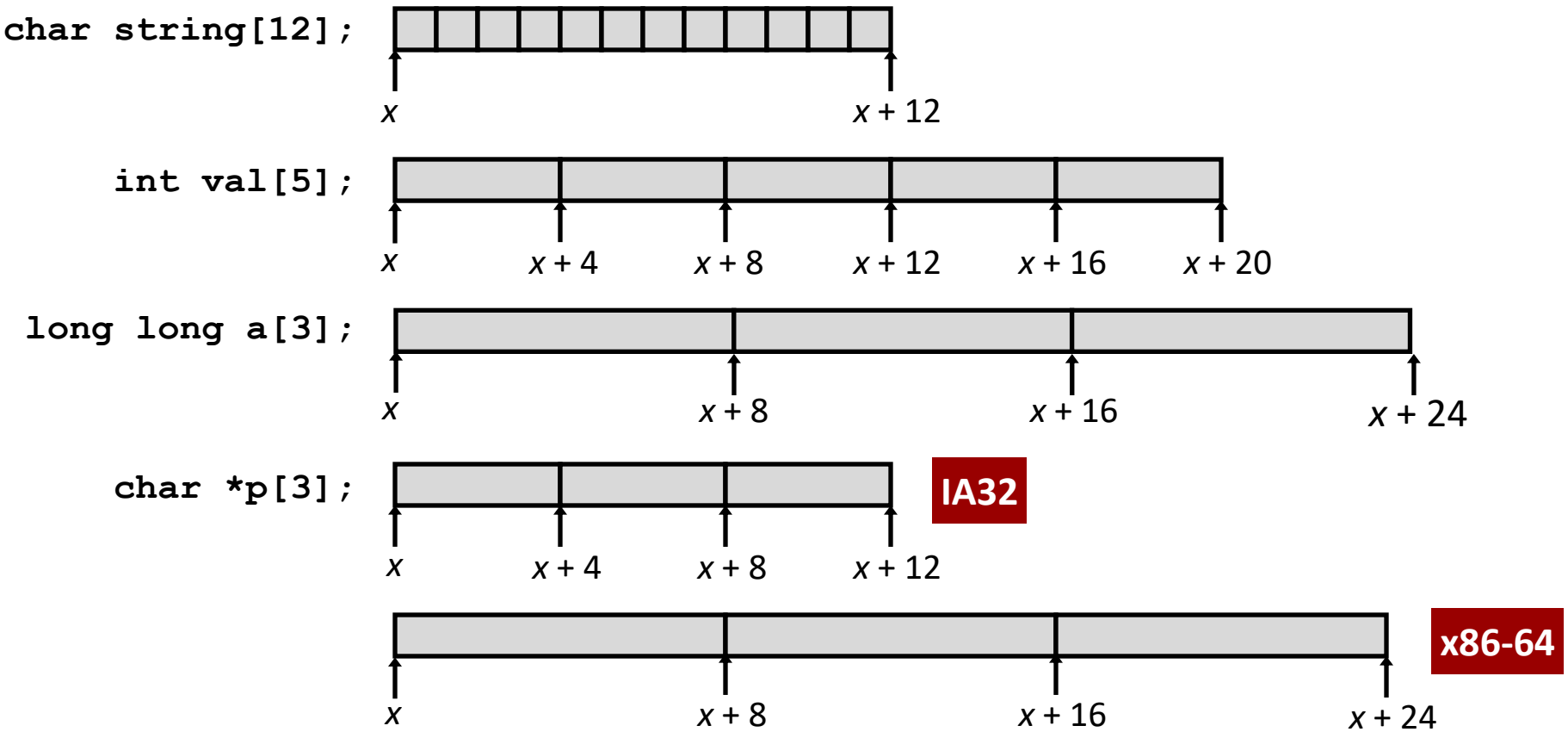
Floating point

- Stored and operated on in floating point registers

Intel	Asm	Bytes	C
Single	`s`	4	`float`
Double	`l`	8	`double`
Extended	`t`	10/12/16	`long double`

Array allocation

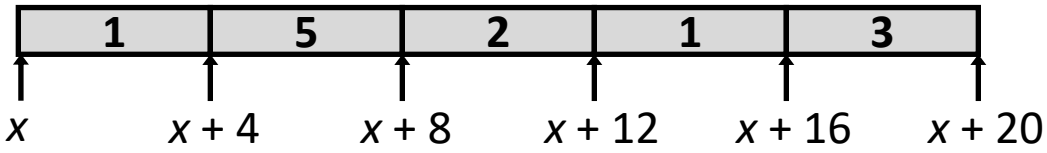
- **Basic principle → *T* `A[L];`**
  - Array of data type *T* and length *L*
  - Contiguously allocated region of *L* * `sizeof` (*T*) bytes



Array access

- **Identifier can be used as a pointer to first element**
  - Type *T**

```
int val[5];
```

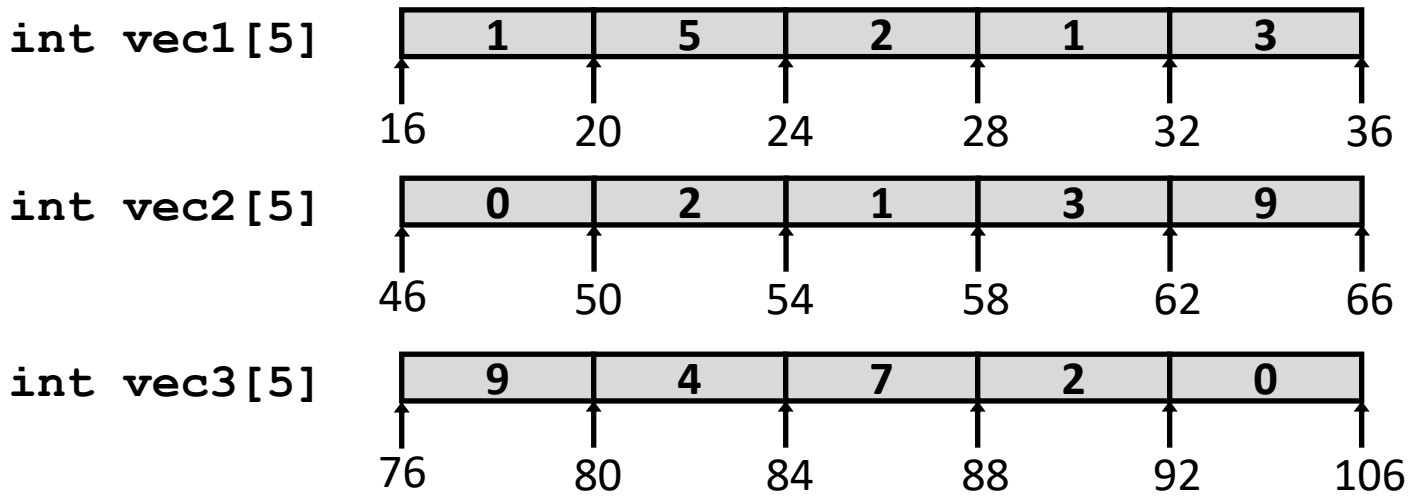


Reference	Type	Value
`val[4]`	`int`	3
`val`	`int *`	*x*
`val+1`	`int *`	*x* + 4
`&val[2]`	`int *`	*x* + 8
`val[5]`	`int`	??
`*(val+1)`	`int`	5
`val + i`	`int *`	*x* + 4 *i*

Array allocation – Example

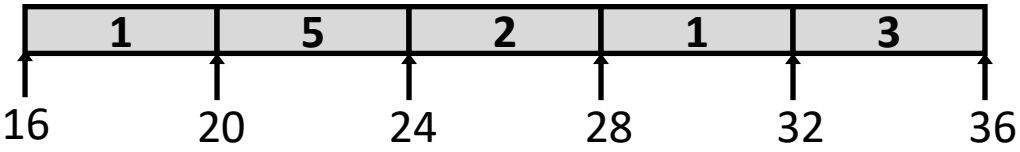
```
#define LEN 5

int vec1[LEN] = { 1, 5, 2, 1, 3 };
int vec2[LEN] = { 0, 2, 1, 3, 9 };
int vec3[LEN] = { 9, 4, 7, 2, 0 };
```



Array access – Example

`int vec1[5]`



C code

```
int get_digit(int *vec1, int dig){
  return vec1[dig];
}
```

Assembly code

```
  movl 8(%ebp), %edx          # vec1
  movl 12(%ebp), %eax         # dig
  movl (%edx,%eax,4),%eax     # vec1[dig]
```

- **`%edx` - starting address of array**
- **`%eax` - array index**
- **Desired digit at `4*%eax + %edx`**
- **Use memory reference `(%edx,%eax,4)`**

Array loop – Example

C code

```
void array_incr(int *vec1){
  int i;
  for (i = 0; i < LEN; i++)
    vec1[i]++;
}
```

Assembly code

```
   movl   8(%ebp), %edx          # vec1
   movl   LEN, %ecx              # LEN
   movl   $0, %eax               # i
.L4:                             # loop:
   addl   $1, (%edx,%eax,4)      #   vec1[i]++
   addl   $1, %eax               #   i++
   cmpl   %ecx, %eax             #   LEN:i
   jne    .L4                    #   if !=, goto loop
```

Pointer loop – Example

C code – int*

```
void array_incr_p(int *v){
  int *end = v + LEN;

  do{
    (*v)++;
    v++;
  }while(v != end);
}
```

C code – char*

```
void array_incr_p(int *v){
  char *ptr = v;
  int i = 0;
  do{
    (*((int *)(ptr+i)))++;
    i += 4;
  }while(i != 4 * LEN);
}
```

Assembly code

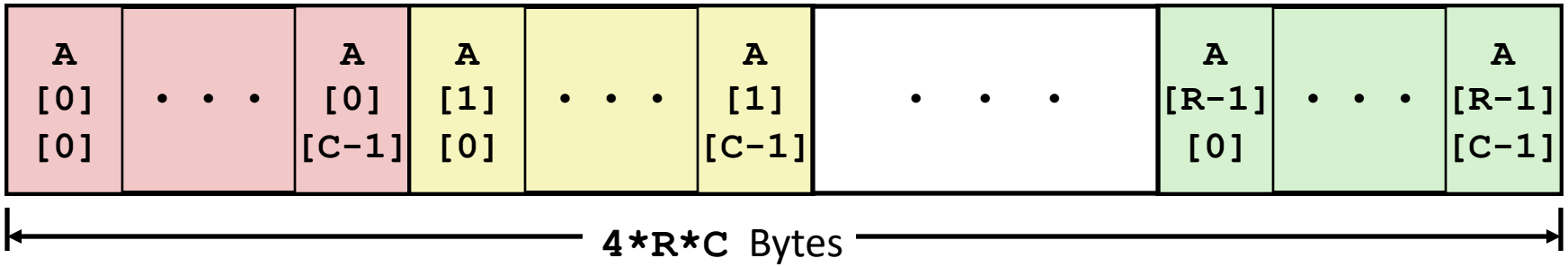
```
   movl   8(%ebp), %edx          # ptr = v
   movl   $0, %eax               # i = 0
.L8:                             # loop:
   addl   $1, (%edx,%eax)        #   Increment ptr+i
   addl   $4, %eax               #   i += 4
   cmpl   $20, %eax              #   Compare i:20
   jne    .L8                    #   if !=, goto loop
```

Multidimensional (nested) arrays

- **Basic principle → *T* `A[R][C]`**
  - 2D array of data type *T*
  - *R* rows, *C* columns
  - Type *T* element requires *K* bytes
- **Array size**
  - *R* * *C* * *K* bytes
- **Arrangement**
  - Row-Major ordering

```
[ A[0][0]    • • •    A[0][C-1]
    •                    •
    •                    •
    •                    •
  A[R-1][0]  • • •  A[R-1][C-1] ]
```

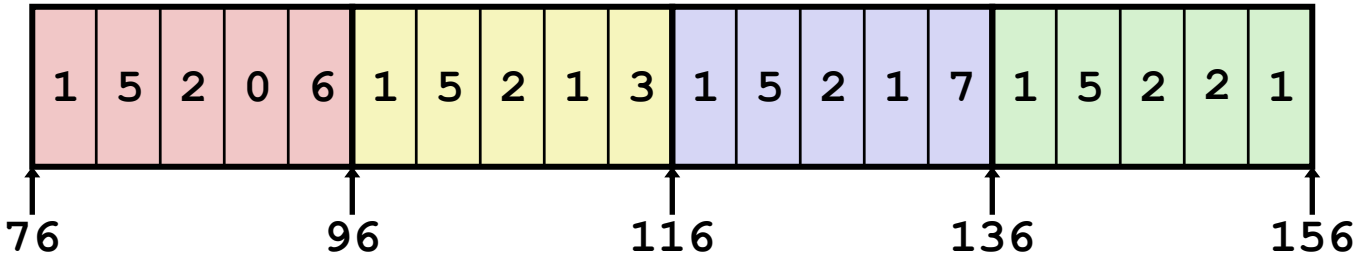
```
int A[R][C];
```



Nested array – Example

```
#define RCOUNT 4
int m[RCOUNT][LEN] =
  {{1, 5, 2, 0, 6},
   {1, 5, 2, 1, 3 },
   {1, 5, 2, 1, 7 },
   {1, 5, 2, 2, 1 }};
```

`int m[4][5];`

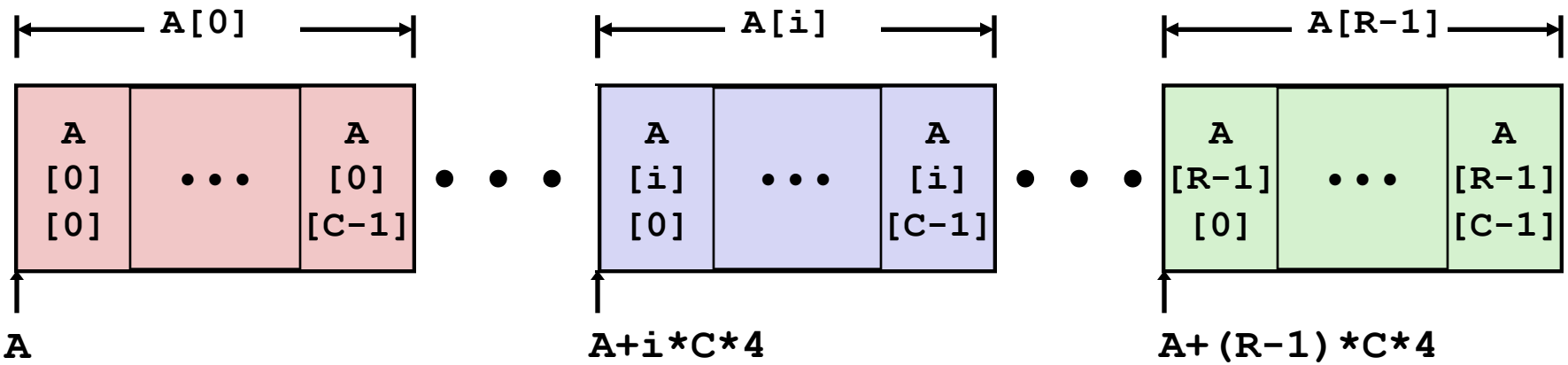


- **Variable `m`**
  - Array of 4 elements, allocated contiguously
  - Each element is an array of 5 **`int`**'s, allocated contiguously
- **"Row-Major" ordering of all elements guaranteed**

Nested array – Row access

- **Row vectors**
  - `A[i]` is array of *C* elements
  - Each element of type *T* requires *K* bytes
  - Starting address: `A` + *i* * (*C* * *K*)

```
int A[R][C];
```



Nested array – Row access example

```
int* get_m(int index){
  return m[index];
}
```

```
array m[RCOUNT][LEN] =
  {{1, 5, 2, 0, 6},
   {1, 5, 2, 1, 3 },
   {1, 5, 2, 1, 7 },
   {1, 5, 2, 2, 1 }};
```

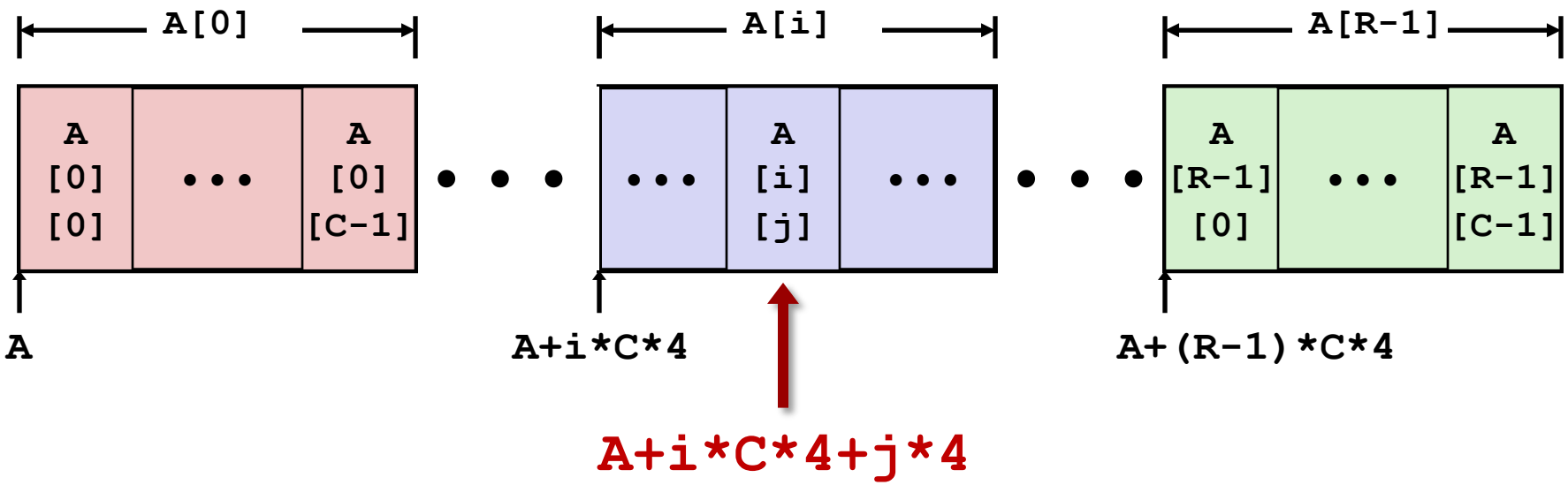
```
movl 8(%ebp), %eax                # index
leal (%eax,%eax,4),%eax           # 5 * index
leal m(,%eax,4),%eax              # m + (20 * index)
```

- **Row vector**
  - **`m[index]`**: array of 5 **`int`**'s
  - Starting address: **`m + 20*index`**
- **IA32 code**
  - Computes and returns address as: **`m + 4*(index + 4*index)`**

Nested array – Element access

- **Array elements**
  - `A[i][j]` is element of type *T*, which requires *K* bytes
  - Address: $A + i * (C * K) + j * K = A + (i * C + j) * K$

```
int A[R][C];
```



Nested array – Element access example

```
int get_m_digit(int index, int dig){
  return m[index][dig];
}
```

```
movl   8(%ebp), %eax              # index
leal   (%eax,%eax,4), %eax        # 5*index
addl   12(%ebp), %eax             # 5*index+dig
movl   m(,%eax,4), %eax           # offset + 4*(5*index+dig)
```

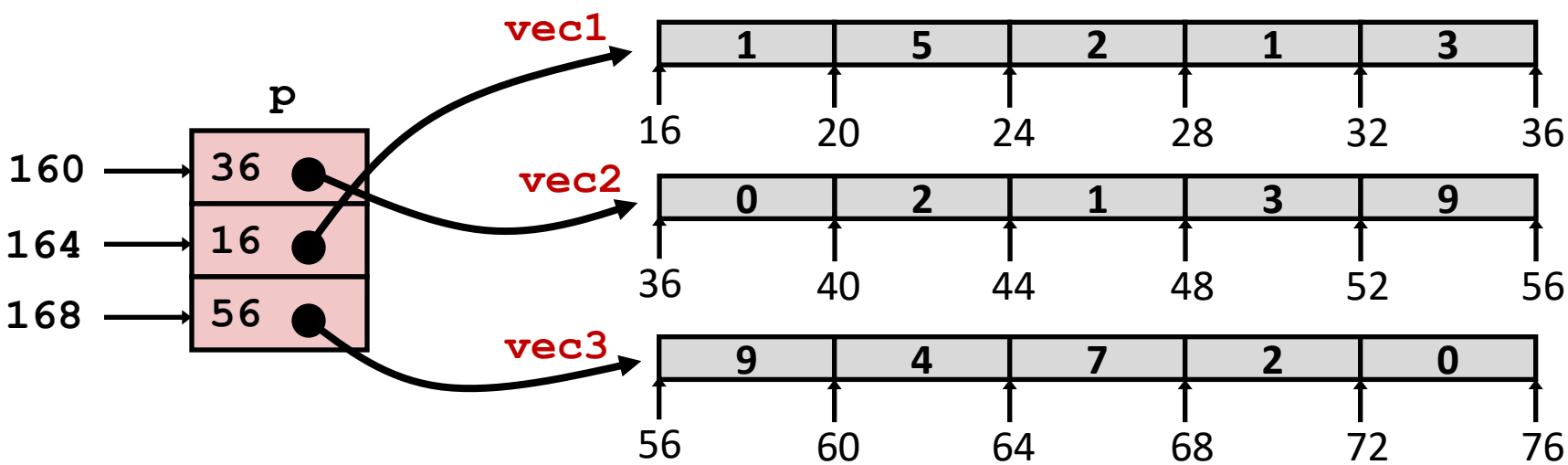
- **Array elements**
  - `m[index][dig]` is `int`
  - Address: `m + 20*index + 4*dig` = `m + 4*(5*index + dig)`
- **IA32 code**
  - Computes address as: `m + 4*((index+4*index)+dig)`

Multi-level array – Example

```
int vec1[LEN] = { 1, 5, 2, 1, 3 };
int vec2[LEN] = { 0, 2, 1, 3, 9 };
int vec3[LEN] = { 9, 4, 7, 2, 0 };
```

```
#define PCOUNT 3
int *p[PCOUNT] = {vec2, vec1, vec3};
```

- Variable `p` denotes array of 3 elements
- Each element is a pointer
- Each pointer points to array of `int`'s



Multi-level array – Element access

```
int get_p_digit(int index, int dig){
  return p[index][dig];
}
```

```
movl   8(%ebp), %eax            # index
movl   p(,%eax,4), %edx         # row = p[index]
movl   12(%ebp), %eax           # dig
movl   (%edx,%eax,4), %eax      # row[dig]
```

- **IA32 code**
  - Element accessed at: `Mem[Mem[p+4*index]+4*dig]`
- **Must do two memory reads**
  - First get pointer to row array
  - Then access element within array

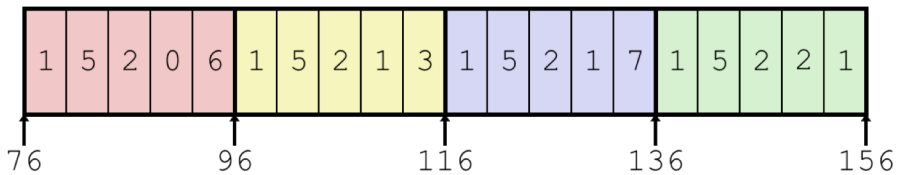
Array element accesses – Comparison

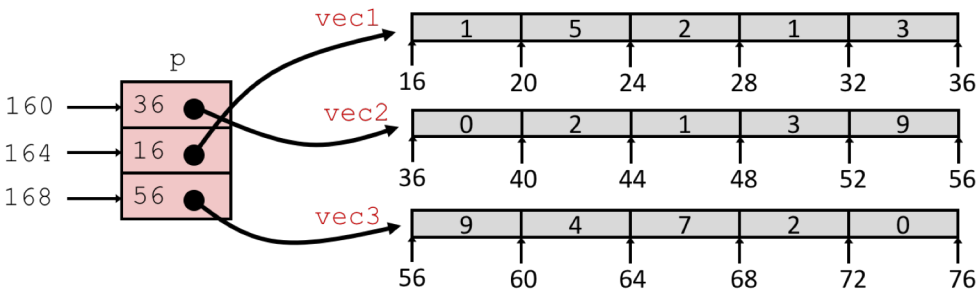
Nested array

```
int get_m_digit(int i, int d){
  return m[i][d];
}
```

Multi-level array

```
int get_p_digit(int i, int d){
  return p[i][d];
}
```





- **Accesses looks similar in C, but addresses are very different**

`Mem[m+20*i+4*d]`

`Mem[Mem[p+4*i]+4*d]`

Today

- Allocation and access
  - One-dimensional
  - Multi-dimensional (nested)
  - Multi-level (pointers)
- **Access optimizations**
  - Fixed-size arrays

Fixed-size arrays

- **It is possible to make many optimizations for code operating on multi-dimensional arrays of fixed size**
  - We (and the compiler) know the value of N at compile time

```
#define N 16

/* Get element a[i][j] */
int fix_elem(int m[N][N], int i, int j){
  return m[i][j];
}
```

16 X 16 Matrix access example

- **Access array elements**
  - Address: **M + *i* * (*C* * *K*) + *j* * *K***
  - C = 16, K = 4

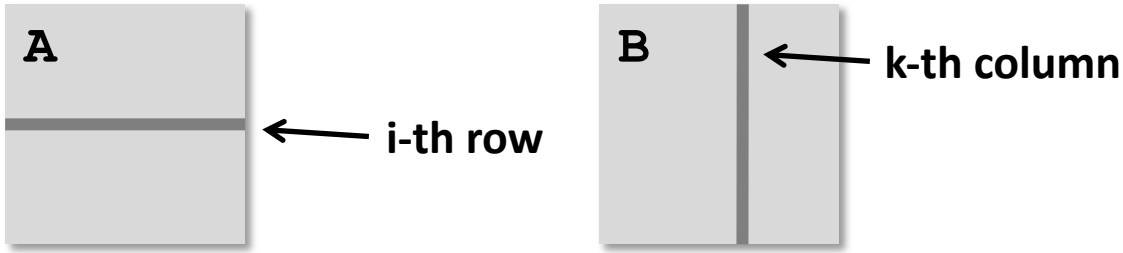
```
/* Get element m[i][j] */
int fix_elem(int m[N][N], int i, int j){
  return m[i][j];
}
```

```
  movl   12(%ebp), %edx          # i
  sall   $6, %edx                # i*64
  movl   16(%ebp), %eax          # j
  sall   $2, %eax                # j*4
  addl   8(%ebp), %eax           # m + j*4
  movl   (%eax,%edx), %eax       # *(m + j*4 + i*64)
```

Optimizing fixed-sized array access

- **Within a loop, index computations can often be optimized by exploiting the regularity of the access patterns**

```
/* Compute i,k of fixed matrix product */
int fix_prod_elem(int A[N][N], int B[N][N], int i, int k){
  int j;
  int result = 0;
  for(j=0; j < N; j++){
    result += A[i][j] * B[j][k];
  }
  return result;
}
```



Optimizing fixed-sized array access

- **The loop will access just the elements of row *i* of array A**
  - Create a local pointer to provide direct access to row *i*
  - Initialize pointer to `&A[i][0]`, and so array element `A[i][j]` can be accessed as `Arow[j]`

- **The loop will access the elements of array B as `B[0][k]`, `B[1][k]`,...,`B[15][k]` in sequence**
  - These elements occupy positions in memory starting with `&B[0][k]` and spaced 64 bytes apart
  - Use a pointer `Bcol` to access these successive locations

Optimized C code

```
/* Compute i,k of fixed matrix product */
int fix_prod_elem_opt(int A[N][N], int B[N][N], int i, int k) {
  int *Arow = &A[i][0];
  int *Bcol = &B[0][k];
  int j;
  int result = 0;

  for(j=0; j != N; j++){
    result += *(Arow+j) * *Bcol;
    Bcol += N;
  }
  return result;
}
```

Optimized Assembly code

```
# %esi = Arow
# %ecx = Bcol
# %edx = j
# %ebx = result
.L6:                                    #loop:
  movl   (%ecx), %eax                   # Get *Bcol
  imull  (%esi,%edx,4), %eax            # Multiply by *(Arow+j)
  addl   %eax, %ebx                     # Add to result
  incl   %edx                           # Increment j
  addl   $64, %ecx                      # Add 64 to Bcol
  cmpl   $16, %edx                      # Compare j:16
  jne    .L6                            # If !=, goto loop
```

- **Machine code considers every pointer to be a byte address**
  - `%ecx` is incremented by 64 within the loop

Summary

- **Arrays**
  - Elements packed into contiguous region of memory
  - Use index arithmetic to locate individual elements

- **Optimizing access within loops**
  - Often we can optimize the index computations by exploiting the regularity of the access patterns